

Analyse numérique et calcul scientifique - 1 TP 2 (3h)

Interpolation de données

1 Objectifs

- Lire, comprendre, utiliser et modifier un code Python plus complexe (scripts, fonctions, classes).
- Se familiariser avec les méthodes d'interpolation par polynômes de Lagrange, de Newton et de Vandermonde.
- Tester numériquement la méthode des noeuds de Chebyshev pour atténuer les erreurs commises par l'interpolation polynomiale de degré élevé.
- Se familiariser avec les méthodes d'interpolation linéaire par morceaux.
- Communiquer des résultats scientifiques.

2 Description des études

Ce TP est composé de cinq études : les trois premières sont très guidées (la structure des codes est déjà fournie), tandis que dans les deux dernières, vous devrez implémenter les méthodes vous-même.

2.1 Interpolation de Lagrange

Dans cette étude, vous devez suivre les instructions du fichier `lagrange.ipynb`. L'objectif principal est d'analyser les erreurs numériques commises dans l'interpolation polynomiale, en particulier lorsqu'elle est calculée avec la base de Lagrange. Vous apprendrez également à structurer une implémentation générale d'une méthode d'interpolation à l'aide d'une classe dédiée.

2.2 Noeuds de Chebyshev

Dans cette étude, vous devez suivre les instructions du fichier `chebyshev.ipynb`. L'objectif principal est d'« améliorer » les résultats de l'étude précédente en choisissant différemment les noeuds d'interpolation.

2.3 Interpolation par morceaux

Dans cette étude, vous devez suivre les instructions du fichier `polynome_morceaux.ipynb`. Elle est similaire à la première étude, mais porte sur l'interpolation linéaire par morceaux.

2.4 Régression versus interpolation

L'objectif de cette étude est de comparer un polynôme interpolant et un polynôme de régression obtenu par la méthode des moindres carrés. Voici un guide des étapes à suivre :

1. Implémenter une fonction qui retourne la matrice de Vandermonde pour un jeu de données donné.

2. Implémenter une fonction qui retourne la valeur d'un polynôme de degré n , $p_n(x)$, en prenant $\mathbf{a} \in \mathbb{R}^{n+1}$ (vecteur des coefficients du polynôme) et x comme arguments.
3. Générer un jeu de données bruité (libre). Une suggestion consiste à partir d'un polynôme donné et à y ajouter un bruit avec la fonction `np.random.normal`.
4. Si la matrice est carrée, résoudre le système linéaire $\mathbf{V}\mathbf{a} = \mathbf{Y}$ avec la fonction `np.linalg.solve` de NumPy.
5. Si la matrice n'est pas carrée, c'est-à-dire si le degré du polynôme est inférieur au nombre de points dans le jeu de données, résoudre le système $(\mathbf{V}^T \mathbf{V})\mathbf{a} = \mathbf{V}^T \mathbf{Y}$ issu de la méthode des moindres carrés.
6. Comparer graphiquement les deux méthodes, en faisant varier les degrés du polynôme pour les deux cas.
7. (Bonus) Pour le cas de l'interpolation, vérifier qu'en augmentant trop le degré du polynôme, le nombre de conditionnement de $\mathbf{V}$ se détériore (et donc les coefficients obtenus par la résolution du système). La fonction `np.linalg.cond` permet de calculer le nombre de conditionnement.

Le code développé doit être nommé `regression_versus_interpolation.ipynb`.

2.5 Équivalence entre les méthodes de Lagrange, Newton et Vandermonde

L'objectif de cette étude est de vérifier que le polynôme interpolant obtenu par les trois méthodes étudiées (Lagrange, Newton et Vandermonde) est effectivement le même. Dans cette étude, vous devez implémenter la méthode de Newton et copier les codes des méthodes de Vandermonde et de Lagrange (déjà implémentées dans les études précédentes). Voici un guide pour implémenter la méthode de Newton :

1. Implémenter une fonction récursive (qui s'appelle elle-même) permettant de calculer les différences divisées de Newton (l'ordre de la différence étant déterminé par le nombre de points fournis).
2. Implémenter une classe `Newton`, à l'exemple de la classe `Lagrange` fournie dans la première étude. Cette classe doit stocker les coefficients du polynôme de Newton, calculés une seule fois lors de l'initialisation.

Le code développé doit être nommé `equivalence_lagrange_newton_vandermonde.ipynb`.

A Interpolation par fonctions spline d'ordre 1

Les fonction de bases spline d'ordre 1 sont des polynômes d'ordre 1 par morceaux s'écrivant:

$$\Phi_i^{\text{Spline-1}}(x) \equiv S_i(x) = \begin{cases} \frac{x - x_{i-1}}{x_i - x_{i-1}}, & \text{si } x \in [x_{i-1}, x]; \\ \frac{x_i - x}{x_i - x_{i+1}}, & \text{si } x \in [x, x_{i+1}]; \\ 0, & \text{sinon.} \end{cases} \quad (1)$$

La fonction d'interpolation est un polynôme d'ordre 1 par morceaux:

$$\varphi^{\text{Spline-1}}(x) \equiv s(x) = \sum_{i=0 \dots N} y_i S_i(x). \quad (2)$$

B Rappel : Travail à rendre

Le travail du TP1 et TP2 devront être envoyé par email (aux deux intervenants!) au plus tard le **02/11/25 23h59**. Le travail à rendre est un dossier comprimé (archive `.zip`, `.rar`, ...) nommé `NOM1_NOM2_GPX_TP12` ($X \in A, B$) et contenant : 1 fichier pdf et un sous-dossier `Numerics` avec codes et figures.

Obs : Optionnellement, vous pouvez m'envoyer (Felipe ROCHA) vos travaux partiels jusqu'au **28/10/25 à 23h59** pour un avis intermédiaire (non individualisé). J'envverrai un courriel le 29/10/25 au matin contenant mes impressions, les principales fautes et les points à surveiller.